

Curso de Automatización con Java, Selenium, JUnit y Gherkin

Clase 8 – Selenium + JUnit

COLEGIO
UNIVERSITARIO



CCC
Centro de Capacitación
Continua

Temario

- ♦ Integración de Selenium con JUnit
- ♦ Organización y estructuración de tests

 **Práctica: login/logout con Page Object Model (POM) básico**

Repaso rápido: Selenium + JUnit + AAA

Selenium:

- Automatiza acciones en el navegador (click, sendKeys, getText, etc.).
- Trabaja con WebDriver (ChromeDriver, FirefoxDriver, etc.).

JUnit:

- Framework de pruebas unitarias.
- Nos da anotaciones (@Test, @BeforeEach, @AfterEach) y asserts.

AAA en tests UI:

- **Arrange:** abrir navegador, ir a la URL, preparar Page Objects, datos.
- **Act:** realizar acciones (login, navegar, etc.).
- **Assert:** verificar resultados (título, URL, mensajes, elementos visibles).

Esto lo vamos a aplicar en un test real.

Estructura de un proyecto de automatización

src

└─ test

└─ java

└─ base

└─ TestBase.java

└─ pages

└─ LoginPage.java

└─ HomePage.java

└─ tests

└─ LoginTest.java

- **base/** → clases compartidas (WebDriver, setup, etc.).
- **pages/** → Page Objects (una clase por página).
- **tests/** → Clases de test (escenarios de negocio).

Clase base: TestBase (setup / teardown de WebDriver)

- Centralizar la configuración de WebDriver.
- No repetir `WebDriverManager` + `new ChromeDriver()` en cada test.
- Usar `@BeforeEach` y `@AfterEach` para abrir/cerrar navegador automáticamente.

```
package base;

import io.github.bonigarcia.wdm.WebDriverManager;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class TestBase {

    protected WebDriver driver;

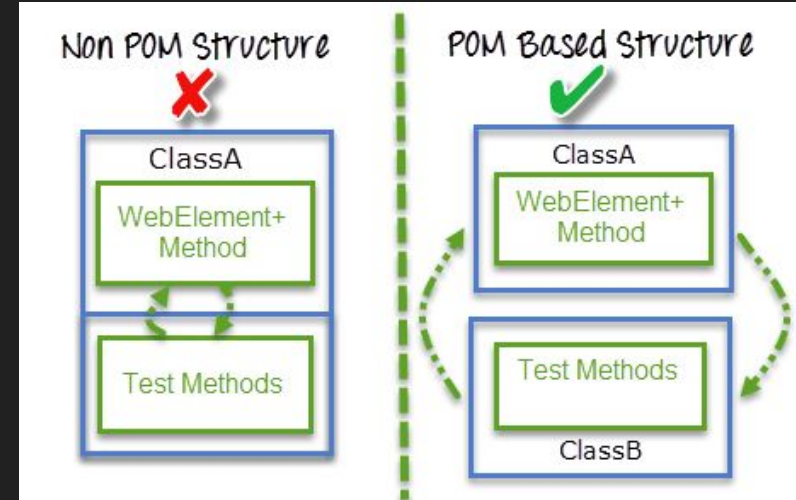
    @BeforeEach
    public void setup() {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }

    @AfterEach
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

Page Object Model (POM)

¿Qué es POM?

- Un patrón donde **cada página** de la aplicación web se modela como una **clase Java**.
- Cada clase contiene:
 - Localizadores de la página.
 - Métodos que representan acciones de usuario (login, logout, buscar, etc.).
- El test no conoce los **By**, solo llama a métodos de alto nivel.



Swag Labs

Username

Password

Login

```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {

    private WebDriver driver;

    // Localizadores
    private By userInput = By.id("user-name");
    private By passInput = By.id("password");
    private By loginBtn = By.id("login-button");

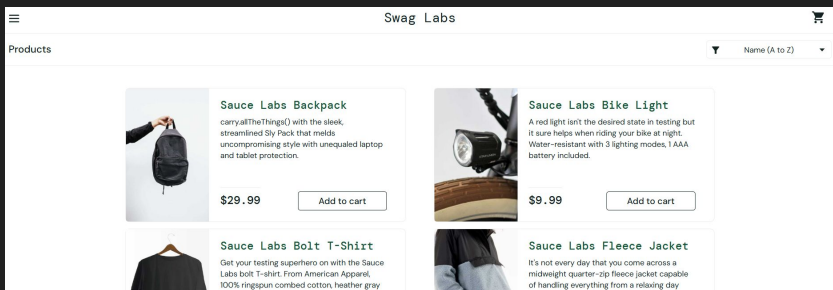
    // Constructor
    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    // Acciones
    public void ingresarUsuario(String usuario) {
        driver.findElement(userInput).clear();
        driver.findElement(userInput).sendKeys(usuario);
    }

    public void ingresarPassword(String password) {
        driver.findElement(passInput).clear();
        driver.findElement(passInput).sendKeys(password);
    }

    public void hacerLogin(String usuario, String password) {
        ingresarUsuario(usuario);
        ingresarPassword(password);
        driver.findElement(loginBtn).click();
    }

    public String obtenerMensajeError() {
        return driver.findElement(errorMessage).getText();
    }
}
```



```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class HomePage {

    private WebDriver driver;

    private By menuBtn    = By.id("react-burger-menu-btn");
    private By logoutBtn  = By.id("logout_sidebar_link");
    private By tituloProductos = By.cssSelector(".title");

    public HomePage(WebDriver driver) {
        this.driver = driver;
    }

    public void logout() {
        driver.findElement(menuBtn).click();
        driver.findElement(logoutBtn).click();
    }

    public String obtenerTituloSeccion() {
        return driver.findElement(tituloProductos).getText();
    }
}
```


Test de login/logout con Selenium + JUnit + POM + AAA

Clase: `LoginTest.java`

Hereda de `TestBase`.

```
package tests;

import base.TestBase;
import org.junit.jupiter.api.Test;
import pages.LoginPage;
import pages.HomePage;

import static org.junit.jupiter.api.Assertions.*;

public class LoginTest extends TestBase {

    @Test
    public void loginLogoutCorrecto() {

        // ARRANGE
        driver.get("https://www.saucedemo.com/");
        LoginPage loginPage = new LoginPage(driver);
        HomePage homePage = new HomePage(driver);

        // ACT
        loginPage.hacerLogin("standard_user", "secret_sauce");

        // ASSERT: después del login
        assertEquals("Products",
            homePage.obtenerTituloSeccion(),
            "El título de la sección no es el esperado luego del login");

        // ACT 2: Logout
        homePage.logout();

        // ASSERT 2: volvemos a la pantalla de login
        assertTrue(driver.getCurrentUrl().contains("saucedemo"),
            "El usuario debería volver a la página de login después del logout");
    }
}
```

Práctica

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

Ejercicio guiado

1. Ejercicio 1 – Login incorrecto

- Crear un test `loginIncorrectoDebeMostrarError()`.
- Usar `LoginPage.hacerLogin()` con usuario o password inválidos.
- Verificar con un assert que el mensaje de error contenga un texto esperado.

2. Ejercicio 2 – Refactorizar duplicación

- Si en los tests hay código repetido para ir a la URL, moverlo a un método helper o al `@BeforeEach` de `TestBase`.

3. Ejercicio 3 – Separar tests positivos y negativos

- Un test para login correcto.
- Uno para login incorrecto por usuario.
- Uno por password.

Cierre

**COLEGIO
UNIVERSITARIO**



CCC

Centro de Capacitación
Continua

Buenas prácticas

- Usar una **clase base** (**TestBase**) para WebDriver.
- No instanciar el **driver** dentro de cada test.
- Mantener los **tests limpios**, con foco en el escenario, no en detalles de UI.
- Los detalles de UI (locators, clicks, sendKeys) viven en los **Page Objects**.
- Usar **AAA** para ordenar el test.
- Agregar mensajes claros en los **asserts**:
 - Qué se esperaba.
 - Qué se está verificando.

¿Preguntas?

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

¡Gracias!

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua